

ASSIGNMENT NO.8: Views and Data Abstraction

Objective:

The objective of this practical is to help students understand the concept of views in MySQL. Students will learn how to create and use views, simplify complex queries using views, understand the concept of updating data through views, and identify practical use cases of views in reporting and data abstraction.

1. Introduction to Views and Data Abstraction

A **view** is a virtual table created from the result of an SQL query. It does not store data separately like a normal table. Instead, it shows data from one or more base tables.

Views are useful because they hide complex query logic and display only the required data to users.

Simple Meaning

Term	Meaning
Table	Stores actual data physically
View	Shows selected data from one or more tables
Base Table	Original table from which view is created
Data Abstraction	Showing only required data and hiding unnecessary details
Virtual Table	A table-like structure that displays query results

Why Views Are Used

Reason	Explanation
Simplicity	Complex queries can be saved as views
Security	Sensitive columns can be hidden
Reusability	Same query can be used again and again
Reporting	Reports can be generated easily
Data abstraction	Users see only required information

2.Creating Sample Tables and Inserting Records

For this practical, we will use two tables: Course and Student.

Course Table Structure

Column Name	Data Type	Constraint
CourseID	INT	PRIMARY KEY
CourseName	VARCHAR(50)	NOT NULL

Fees	DECIMAL(10,2)	NOT NULL
------	---------------	----------

Student Table Structure

Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY
StudentName	VARCHAR(50)	NOT NULL
CourseID	INT	FOREIGN KEY
City	VARCHAR(30)	
Column Name	Data Type	Constraint
Marks	INT	
Email	VARCHAR(60)	

MySQL Code to Create Database and Tables

```
CREATE DATABASE CollegeDB;
```

```
USE CollegeDB;
```

```
CREATE TABLE Course (CourseID INT PRIMARY KEY, CourseName VARCHAR(50) NOT NULL, Fees DECIMAL(10,2) NOT NULL );
```

```
CREATE TABLE Student ( StudentID INT PRIMARY KEY, StudentName VARCHAR(50) NOT NULL, CourseID INT, City VARCHAR(30), Marks INT, Email VARCHAR(60), FOREIGN KEY (CourseID) REFERENCES Course(CourseID));
```

MySQL Code to Insert Records

```
INSERT INTO Course
(CourseID, CourseName, Fees)
VALUES
(101, 'BSc IT', 45000.00),
(102, 'BCA', 40000.00),
(103, 'BSc CS', 42000.00),
(104, 'Data Science', 55000.00);
```

```
INSERT INTO Student
(StudentID, StudentName, CourseID, City, Marks, Email)
VALUES
(1, 'Rahul Sharma', 101, 'Mumbai', 78, 'rahul@gmail.com'),
(2, 'Priya Patil', 102, 'Pune', 85, 'priya@gmail.com'),
(3, 'Aman Verma', 103, 'Nashik', 67, 'aman@gmail.com'),
(4, 'Sneha Joshi', 101, 'Mumbai', 91, 'sneha@gmail.com'),
(5, 'Karan Mehta', 102, 'Thane', 74, 'karan@gmail.com'),
(6, 'Neha Shah', 101, 'Pune', 88, 'neha@gmail.com'),
(7, 'Rohan Desai', 103, 'Mumbai', 69, 'rohan@gmail.com'),
(8, 'Meera Kulkarni', 102, 'Nashik', 82, 'meera@gmail.com'),
(9, 'Arjun Nair', 101, 'Thane', 59, 'arjun@gmail.com'),
(10, 'Divya More', 102, 'Mumbai', 93, 'divya@gmail.com');
```

Course Table

CourseID	CourseName	Fees
101	BSc IT	45000.00
102	BCA	40000.00
103	BSc CS	42000.00
104	Data Science	55000.00

Student Table

StudentID	StudentName	CourseID	City	Marks	Email
1	Rahul Sharma	101	Mumbai	78	rahul@gmail.com
2	Priya Patil	102	Pune	85	priya@gmail.com
3	Aman Verma	103	Nashik	67	aman@gmail.com
4	Sneha Joshi	101	Mumbai	91	sneha@gmail.com
5	Karan Mehta	102	Thane	74	karan@gmail.com
6	Neha Shah	101	Pune	88	neha@gmail.com
7	Rohan Desai	103	Mumbai	69	rohan@gmail.com
StudentID	StudentName	CourseID	City	Marks	Email
8	Meera Kulkarni	102	Nashik	82	meera@gmail.com
9	Arjun Nair	101	Thane	59	arjun@gmail.com
10	Divya More	102	Mumbai	93	divya@gmail.com

3. Creating and Using Views

A view is created using the CREATE VIEW command. Once a view is created, it can be used like a normal table with the SELECT statement.

Syntax

```
CREATE VIEW view_name AS SELECT column1, column2 FROM table_name WHERE condition;
```

Example 1: Create a View for Student Basic Details

```
CREATE VIEW StudentBasicView AS SELECT StudentID, StudentName, City, Marks FROM Student;
```

Use the View

```
SELECT * FROM StudentBasicView;
```

Output

StudentID	StudentName	City	Marks
1	Rahul Sharma	Mumbai	78

2	Priya Patil	Pune	85
3	Aman Verma	Nashik	67
4	Sneha Joshi	Mumbai	91
5	Karan Mehta	Thane	74
6	Neha Shah	Pune	88
7	Rohan Desai	Mumbai	69
8	Meera Kulkarni	Nashik	82
9	Arjun Nair	Thane	59
10	Divya More	Mumbai	93

Explanation

SQL Part	Purpose
CREATE VIEW StudentBasicView	Creates a view named StudentBasicView
SELECT StudentID, StudentName, City, Marks	Selects only required columns
FROM Student	Gets data from Student table
SELECT * FROM StudentBasicView	Displays data from the view

4. Simplifying Complex Queries using Views

Views are very useful when a query uses multiple tables, joins, conditions, or calculations. Instead of writing the same complex query repeatedly, we can create a view once and use it many times.

Complex Query Without View

```
SELECT Student.StudentID, Student.StudentName, Student.City, Student.Marks,
Course.CourseName, Course.Fees FROM Student INNER JOIN Course ON
Student.CourseID = Course.CourseID;
```

Create View for Student Course Details

```
CREATE VIEW StudentCourseView AS SELECT Student.StudentID,
Student.StudentName, Student.City, Student.Marks, Course.CourseName,
Course.Fees FROM Student INNER JOIN Course ON Student.CourseID =
Course.CourseID;
```

Use the View

```
SELECT * FROM StudentCourseView;
```

Output

StudentID	StudentName	City	Marks	CourseName	Fees
1	Rahul Sharma	Mumbai	78	BSc IT	45000.00
2	Priya Patil	Pune	85	BCA	40000.00
3	Aman Verma	Nashik	67	BSc CS	42000.00

4	Sneha Joshi	Mumbai	91	BSc IT	45000.00
5	Karan Mehta	Thane	74	BCA	40000.00
6	Neha Shah	Pune	88	BSc IT	45000.00
7	Rohan Desai	Mumbai	69	BSc CS	42000.00
8	Meera Kulkarni	Nashik	82	BCA	40000.00
9	Arjun Nair	Thane	59	BSc IT	45000.00
10	Divya More	Mumbai	93	BCA	40000.00

Benefit of View

Without View	With View
Complex join query must be written again and again	Simple SELECT * FROM StudentCourseView can be used
More chances of mistakes	Query becomes easier and cleaner
Difficult for beginners	Easy to use like a table
Longer query	Shorter query

5. Filtering and Reporting using Views

After creating a view, we can apply WHERE, ORDER BY, and other clauses on the view. This helps in report generation.

Example 1: Display Students from Mumbai using View

```
SELECT StudentName, City, CourseName, Marks FROM StudentCourseView WHERE City = 'Mumbai';
```

Output

StudentName	City	CourseName	Marks
Rahul Sharma	Mumbai	BSc IT	78
Sneha Joshi	Mumbai	BSc IT	91
Rohan Desai	Mumbai	BSc CS	69
Divya More	Mumbai	BCA	93

Example 2: Display Students Sorted by Marks

```
sql id="au5x8b" SELECT StudentName, CourseName, Marks FROM StudentCourseView ORDER BY Marks DESC;
```

Output

StudentName	CourseName	Marks
Divya More	BCA	93
Sneha Joshi	BSc IT	91
StudentName	CourseName	Marks

Neha Shah	BSc IT	88
Priya Patil	BCA	85
Meera Kulkarni	BCA	82
Rahul Sharma	BSc IT	78
Karan Mehta	BCA	74
Rohan Desai	BSc CS	69
Aman Verma	BSc CS	67
Arjun Nair	BSc IT	59

Example 3: Course-Wise Report using View

```
SELECT CourseName, COUNT(*) AS TotalStudents, AVG(Marks) AS AverageMarks
FROM StudentCourseView GROUP BY CourseName;
```

Output

CourseName	TotalStudents	AverageMarks
BSc IT	4	79.0000
BCA	4	83.5000
BSc CS	2	68.0000

Explanation

Clause	Purpose
WHERE	Filters records from the view
ORDER BY	Sorts records from the view
GROUP BY	Creates summary reports from the view
AVG()	Calculates average marks
COUNT()	Counts total records

6. Updating Data using Views: Conceptual Understanding

In MySQL, some views can be updated, but not all views are updateable. If a view is based on a single table and does not contain complex operations, then it may allow INSERT, UPDATE, or DELETE.

However, views that contain joins, aggregate functions, grouping, or calculated columns are generally not suitable for direct updates.

Updateable and Non-Updateable Views

Type of View	Can Be Updated?	Reason
View based on one table	Usually Yes	Directly maps to one base table
View with selected columns from one table	Usually Yes	MySQL can identify base rows

View using JOIN	Usually Limited/No	Data comes from multiple tables
View using GROUP BY	No	Data is summarized
View using COUNT, AVG, SUM	No	Data is calculated
View using DISTINCT	No	Duplicate rows are removed

Example: Create Simple Updateable View

```
CREATE VIEW StudentMarksView AS SELECT StudentID, StudentName, Marks FROM Student;
```

Update Data using View

```
UPDATE StudentMarksView SET Marks = 80 WHERE StudentID = 1;
```

Check Updated Data from Base Table

```
SELECT StudentID, StudentName, Marks FROM Student WHERE StudentID = 1;
```

Output

StudentID	StudentName	Marks
1	Rahul Sharma	80

Important Concept

Point	Explanation
View does not store separate data	It shows data from base table
Updating a simple view updates base table	The actual table value changes
Complex views may not allow updates	MySQL cannot safely identify rows
Views should be updated carefully	Wrong update can affect original table

7. Use Cases of Views in Reporting

Views are commonly used in reporting because they simplify repeated queries and show only required information. They are also helpful for teachers, administrators, accountants, and management users who need reports without writing complex SQL queries.

Reporting Use Cases

Use Case	View Purpose
Student report	Display student name, city, course, and marks
Course report	Show course-wise student details
Performance report	Display marks and ranking-related data
Fee report	Display student and course fee details
City-wise report	Filter students by city
Security report	Hide sensitive columns such as email

Management dashboard	Show summarized student/course information
----------------------	--

Example: Create View for Management Report

```
CREATE VIEW ManagementReportView AS SELECT Course.CourseName,
COUNT(Student.StudentID) AS TotalStudents,      AVG(Student.Marks) AS
AverageMarks,      MIN(Student.Marks) AS LowestMarks,      MAX(Student.Marks)
AS HighestMarks FROM Course LEFT JOIN Student ON Course.CourseID =
Student.CourseID GROUP BY Course.CourseName;
```

Use the Report View

```
SELECT * FROM ManagementReportView;
```

Output

CourseName	TotalStudents	AverageMarks	LowestMarks	HighestMarks
BSc IT	4	79.0000	59	91
BCA	4	83.5000	74	93
CourseName	TotalStudents	AverageMarks	LowestMarks	HighestMarks
BSc CS	2	68.0000	67	69
Data Science	0	NULL	NULL	NULL

Final Summary Table

Concept	MySQL Command	Purpose
Create view	CREATE VIEW	Creates a virtual table
Use view	SELECT * FROM view_name	Displays view data
Simplify query	View with JOIN	Avoids repeated complex query
Filter view	WHERE	Filters view output
Sort view	ORDER BY	Arranges view output
Report view	GROUP BY, aggregate functions	Creates summary report
Update simple view	UPDATE view_name	Updates base table through view
Remove view	DROP VIEW view_name	Deletes view definition

Drop View Example DROP
VIEW StudentBasicView;

Questions:

Q1) Create a MySQL database named CollegeDB and create two tables: Course and Student.

```
CREATE DATABASE CollegeDB;
```

```
USE CollegeDB;
```

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50) NOT NULL,  
    Fees DECIMAL(10,2) NOT NULL  
);
```

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50) NOT NULL,  
    CourseID INT,  
    City VARCHAR(30),  
    Marks INT,  
    Email VARCHAR(60),  
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

```
INSERT INTO Course VALUES  
(101,'BSc IT',45000.00),  
(102,'BCA',40000.00),  
(103,'BSc CS',42000.00),  
(104,'Data Science',55000.00);
```

```
INSERT INTO Student VALUES  
(1,'Rahul Sharma',101,'Mumbai',78,'rahul@gmail.com'),  
(2,'Priya Patil',102,'Pune',85,'priya@gmail.com'),  
(3,'Aman Verma',103,'Nashik',67,'aman@gmail.com'),  
(4,'Sneha Joshi',101,'Mumbai',91,'sneha@gmail.com'),  
(5,'Karan Mehta',102,'Thane',74,'karan@gmail.com'),  
(6,'Neha Shah',101,'Pune',88,'neha@gmail.com'),  
(7,'Rohan Desai',103,'Mumbai',69,'rohan@gmail.com'),
```

(8,'Meera Kulkarni',102,'Nashik',82,'meera@gmail.com'),
(9,'Arjun Nair',101,'Thane',59,'arjun@gmail.com'),
(10,'Divya More',102,'Mumbai',93,'divya@gmail.com');

Course Table

Column Name	Data Type	Constraint
CourseID	INT	PRIMARY KEY
CourseName	VARCHAR(50)	NOT NULL
Fees	DECIMAL(10,2)	NOT NULL

Student Table

Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY
StudentName	VARCHAR(50)	NOT NULL
CourseID	INT	FOREIGN KEY
City	VARCHAR(30)	
Marks	INT	
Email	VARCHAR(60)	

Insert at least 4 course records and 10 student records.

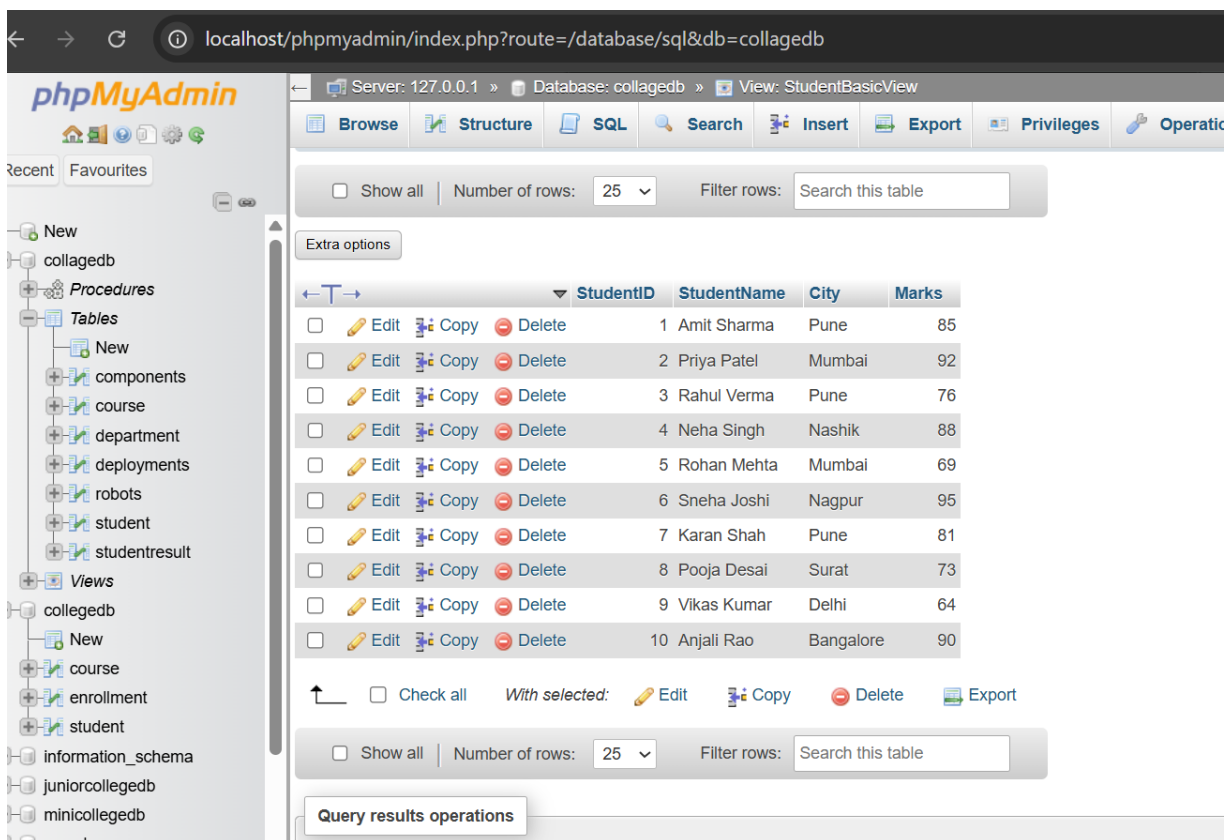
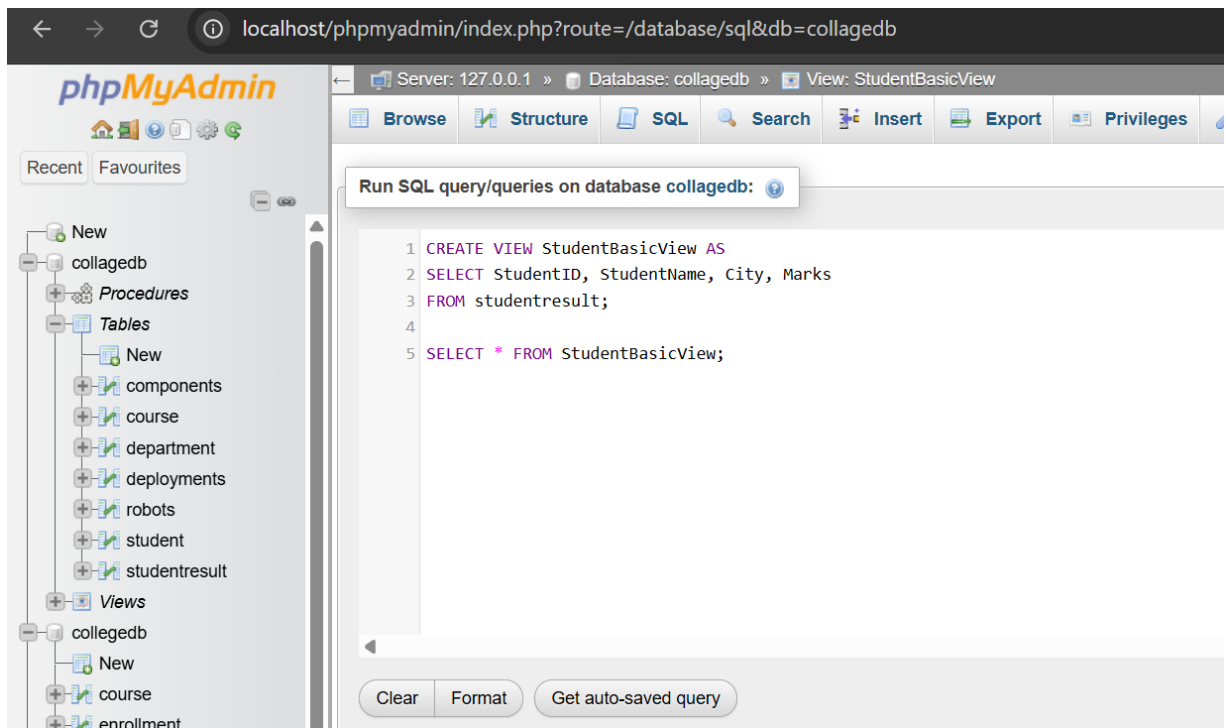
Q2) Create a view named StudentBasicView to display only selected student details.

```
CREATE VIEW StudentBasicView AS
```

```
SELECT StudentID, StudentName, City, Marks
```

```
FROM Studentresult;
```

```
SELECT * FROM StudentBasicView;
```



The view should contain:

Column
StudentID
StudentName

Column
City
Marks

Q3) Create a view named StudentCourseView using Student and Course tables.

```
CREATE VIEW StudentCourseView AS
```

```
SELECT S.StudentID, S.StudentName, S.City, S.Marks,
```

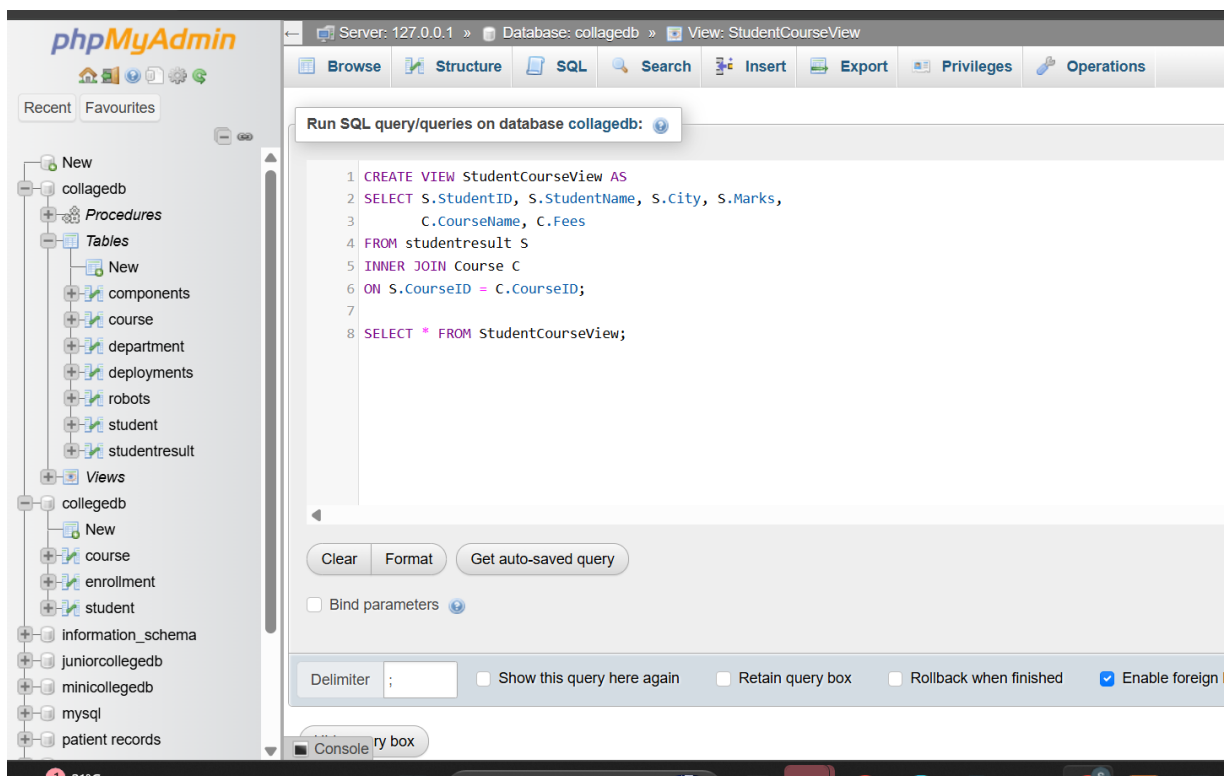
```
       C.CourseName, C.Fees
```

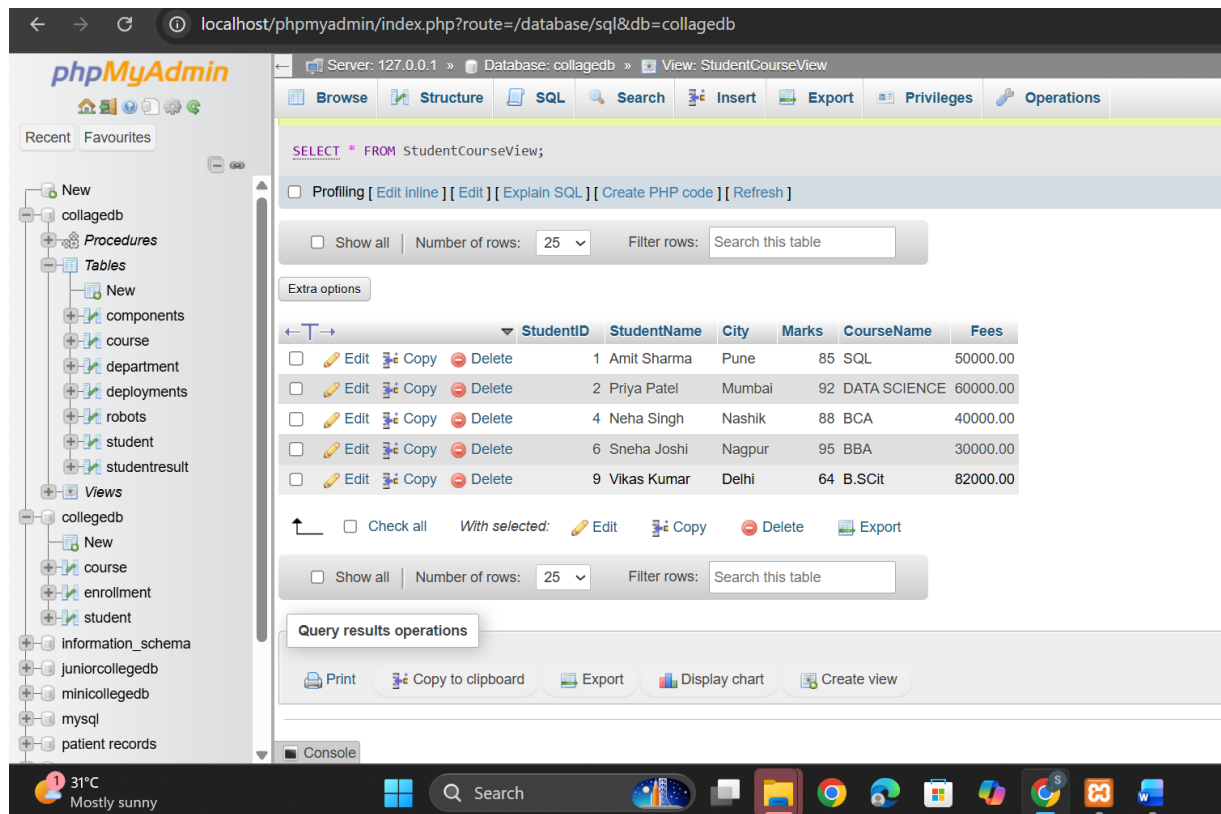
```
FROM Studentresult S
```

```
INNER JOIN Course C
```

```
ON S.CourseID = C.CourseID;
```

```
SELECT * FROM StudentCourseView;
```





The view should display:

Column
StudentID
StudentName
City
Marks
CourseName
Fees

Q4) Write a MySQL query using StudentCourseView to display students from Mumbai.
The output should contain:

```
SELECT StudentName, City, CourseName, Marks
FROM StudentCourseView
WHERE City = 'Mumbai';
```

phpMyAdmin

Server: 127.0.0.1 » Database: collagedb » View: StudentCourseView

Browse Structure SQL Search Insert Export Privileges

Show query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are no

✓ Showing rows 0 - 0 (1 total, Query took 0.0021 seconds.)

```
SELECT StudentName, City, CourseName, Marks FROM StudentCourseView WHERE City = 'Mumbai';
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

StudentName	City	CourseName	Marks
Priya Patel	Mumbai	DATA SCIENCE	92

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

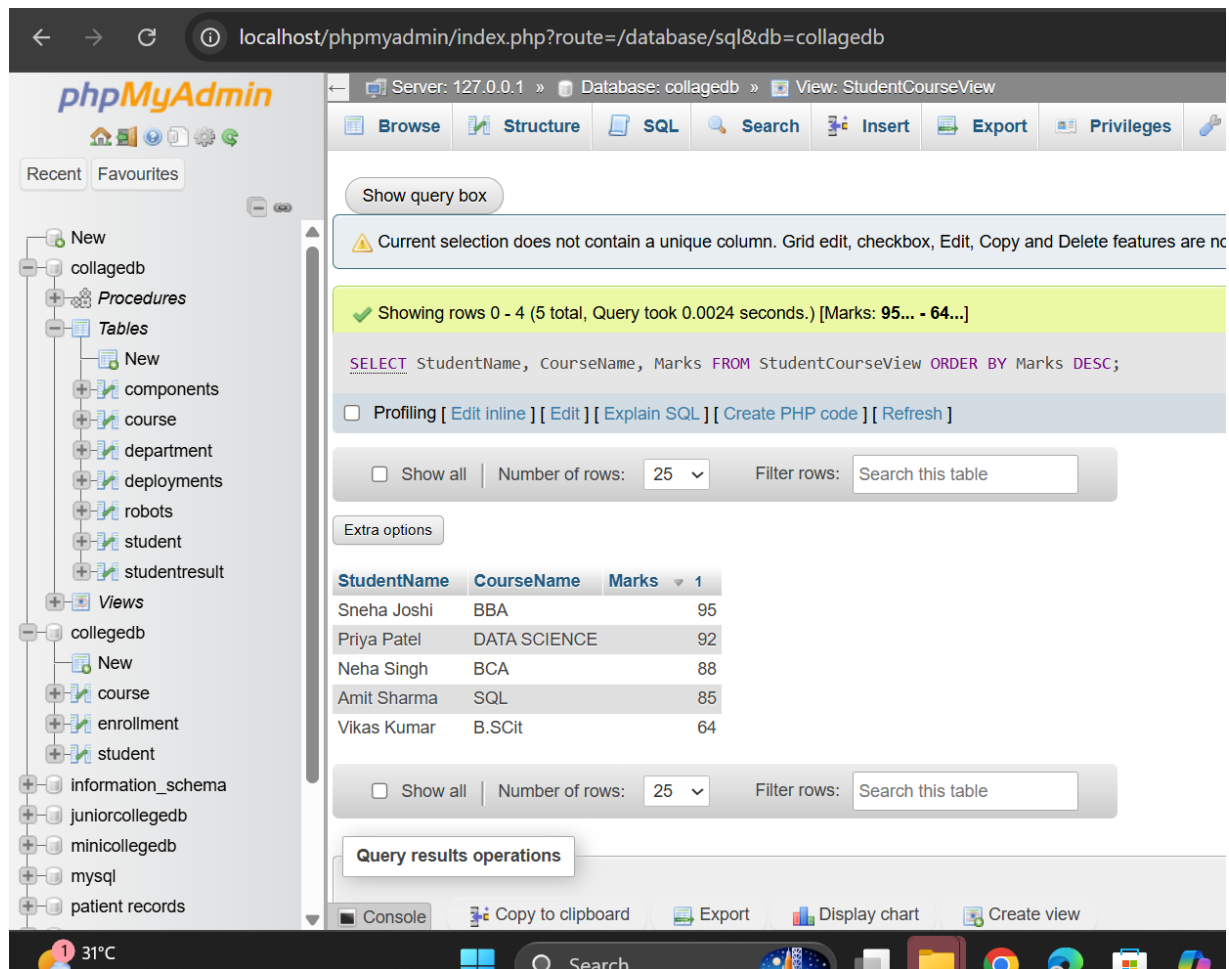
Column
StudentName
City
CourseName
Marks

Q5) Write a MySQL query using StudentCourseView to display students sorted by marks in descending order.

SELECT StudentName, CourseName, Marks

FROM StudentCourseView

ORDER BY Marks DESC;



The output should contain:

Column
StudentName
CourseName
Marks

Q6) Create a report view named ManagementReportView.

CREATE VIEW ManagementReportView AS

SELECT C.CourseName,

COUNT(S.StudentID) AS TotalStudents,

AVG(S.Marks) AS AverageMarks,

MIN(S.Marks) AS LowestMarks,

MAX(S.Marks) AS HighestMarks

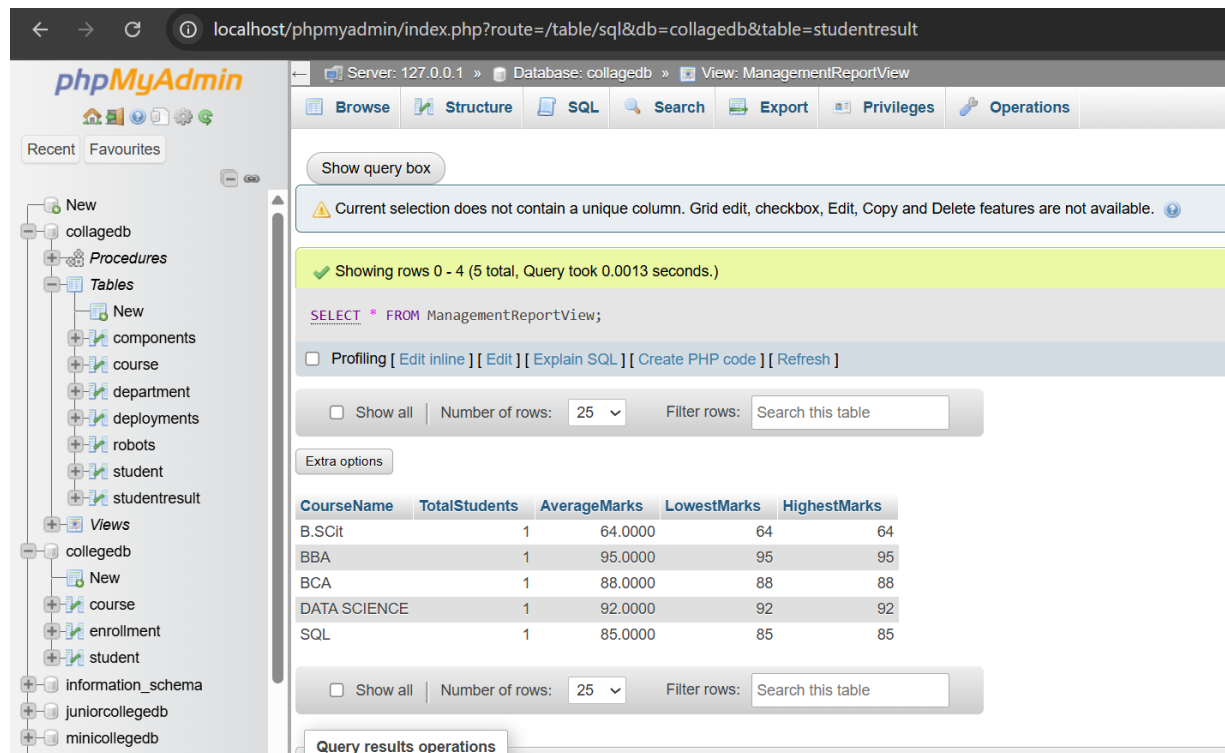
FROM Course C

LEFT JOIN Student S

ON C.CourseID = S.CourseID

GROUP BY C.CourseName;

SELECT * FROM ManagementReportView;



The screenshot shows the phpMyAdmin interface for a database named 'collegedb'. The 'ManagementReportView' view is selected. The query box contains the SQL statement: `SELECT * FROM ManagementReportView;`. The query results are displayed in a table with 5 columns: CourseName, TotalStudents, AverageMarks, LowestMarks, and HighestMarks. The results show data for five courses: B.SCit, BBA, BCA, DATA SCIENCE, and SQL.

CourseName	TotalStudents	AverageMarks	LowestMarks	HighestMarks
B.SCit	1	64.0000	64	64
BBA	1	95.0000	95	95
BCA	1	88.0000	88	88
DATA SCIENCE	1	92.0000	92	92
SQL	1	85.0000	85	85

The view should display course-wise summary:

Column
CourseName
TotalStudents
AverageMarks
LowestMarks
HighestMarks

Also write one line explaining why such a view is useful in reporting.

Assignment Evaluation

- 0: Not Done []
- 1: Incomplete []
- 2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well Done []

Signature of Instructor: _____

